COS 214 Final Project
TIVIDY: Workflow Management System

Academic year: 2026
Demo dates: 2 and 3 November 2026
Demo duration: 30 minutes per group
Team size: 5 or 7 students
Language: C++11

1 Introduction

Practical 6 was the **research and initial design phase** of this same project. Your group must now extend that design, incorporate relevant feedback, implement the system, test it and demonstrate it. The final project is not a separate system or a new start.

Your group must design and implement **TIVIDY**, a generic workflow management system in C++11. The system must define workflows, create independent running workflow instances, coordinate work and participants, manage state and routing decisions, and provide a way to inspect progress and history. There is no prescribed class structure: the design must follow from your research and requirements.

A **workflow definition** describes permitted activities, decisions and transitions. A **workflow instance** represents one independent execution of that definition with its own data, participants, progress and history. Changes in one instance must not unintentionally affect other instances.

2 Two Industry Scenarios

For this project, a **scenario means an industry or application domain**, not a particular department, job function or building. You must implement **two workflows from two distinct industries**. You may retain the industry selected in Practical 6 and add a second industry.

Valid: two different industries	Not valid: one industry split into departments
Healthcare: hospital admission; Legal: case-intake approval	Healthcare: hospital admission; Healthcare: radiology requests
Manufacturing: production order; Insurance: claim assessment	Hospital emergency department; hospital laboratory
Education: university application; Logistics: shipment exception	Retail stock receiving; retail replenishment

Different organisations, work types or departments do not automatically constitute different industries. If unsure whether your pairing qualifies, confirm it with your tutor or AL. Both scenarios must use the **same reusable workflow engine**. Industry-specific definitions, rules, data and adapters are permitted. Two independent hard-coded programs are not.

2.1 Required behaviour in each scenario

- Explain the industry, process, participants, input data and expected result.
- Create or select a workflow definition and start an independent workflow instance.
- Execute several meaningful work items, including dependencies and at least one routing or decision branch.
- Demonstrate a completion path and an exception path (for example rejection, invalid action, failure, cancellation or escalation).
- Show current work, state changes and execution history or audit events.
- Identify the generic engine classes that both scenarios reuse without modification.

3 System Scope and Behaviour

TIVIDY must be an integrated workflow-management system rather than a collection of unrelated GoF demonstrations. The following responsibilities must be addressed in the design and, where relevant, executable in the implementation:

- Workflow definitions and independent instances; stages, work items and their lifecycle.
- Participants, roles, responsibility, work assignment and routing.
- Dependencies, decisions, branching and progression through work.
- Approvals, rejection or failure, and appropriate recovery or escalation behaviour.
- Events or triggers, state changes and inspectable execution history.
- Separation of industry-specific rules from generic workflow-engine responsibilities.

Parallel execution, nested work, external integrations, versioning and persistent storage may strengthen a justified design. However, they are not individually prescribed technologies. Claimed compulsory behaviour must actually execute; a diagram alone is insufficient.

4 Design and UML Documentation

4.1 Gang of Four design patterns

Your Practical 6 submission already introduced your design patterns and their roles. **Review and update that work only where necessary** to match your final design and implementation. You do not need to start the pattern analysis again.

The final working system must still contain at least **10 distinct, meaningful GoF design patterns**. Include at least two patterns from each category (creational, structural and behavioural). At least five must come from the following list: Observer, Iterator, Mediator, Command, Adapter, Chain of Responsibility, Builder, Interpreter, Bridge, Facade, Visitor, Proxy, Singleton and Flyweight.

Ensure that your existing pattern justifications, GoF participant mappings and pattern-specific UML Class Diagrams describe the actual C++ classes and their collaborations. Revise any material that no longer matches the system. Patterns must solve real workflow problems and be executed within the coherent TIVIDY system, not exist as isolated examples. Avoid large centralised type-checking branches where polymorphism is appropriate.

4.2 Requirements and diagrams

You have already developed the initial system scope, requirements, UML diagrams and design decisions in Practical 6. **Reuse and update these deliverables where necessary**; do not recreate them solely for the final submission. The final documents must accurately reflect the implemented system and both industries.

Retain the Practical 6 diagram set: three substantial UML Activity Diagrams (generic execution, another generic engine process and a selected scenario), a complete system UML Class Diagram, at least two UML Sequence Diagrams, and at least one UML State Diagram. Add or modify diagrams as needed to explain the second industry's workflow, the actual runtime behaviour and the final design. Maintain the colour legend, clear ownership and relationships, and documented design decisions and revisions.

Update functional requirements to match the features that you built, and show how the important requirements connect to the design and implementation. You do not need to introduce new non-functional requirements specifically for the final project; if you included them in your earlier design, update them when relevant. All UML diagrams must agree with each other and with the submitted C++ implementation. Supply readable exports and editable Visual Paradigm project files.

5 Implementation and User Interface

Implement the actual workflow engine in **C++11**. The minimum is a **genuinely interactive text-based user interface (TUI)**. A **GUI is preferred**, but **does not earn bonus marks**; a strong TUI can receive full core implementation marks. A GUI must still call the C++ engine, not replace it with a separately hard-coded workflow simulation.

A marker must be able to start an instance, inspect available work, take actions, observe routing and state changes, view history, and use both industries through the running interface. Printed menus without meaningful interaction, pre-recorded videos and slides are not implementation substitutes. Include repeatable build and run instructions. Use realistic test data and reasonable error handling for invalid transitions. External services may be represented by documented test adapters or stubs where necessary.

6 Development Practices

6.1 Compulsory requirements

- **Git and GitHub:** use meaningful commits throughout the project, a documented branching/review process, at least **10 meaningful commits per member** and at least **three closed pull requests** across the group. Every member must contribute genuinely.
- **Code documentation:** document significant classes, operations and non-obvious logic; use **Doxygen**. Provide a README with dependencies, build, run and test instructions for both industries.
- **Automated unit tests:** use a real C++ test framework, such as **GoogleTest, Catch2 or doctest**. Test important workflow behaviour, including transitions, routing, instance isolation and invalid operations. Every member must contribute tests. Full coverage is not required. Manual tests and standalone `assert` programs alone are not sufficient.

6.2 Optional bonus work

You may earn bonus credit for **well-executed technical improvements that go beyond the core requirements**. The opportunities are deliberately open-ended: choose additions that genuinely improve the reliability, maintainability, portability, observability or usability of TIVIDY. Explain why the additions matter and demonstrate that they work.

Examples include a CI/CD pipeline that builds and runs the C++ test suite, a Dockerised application, automated quality checks, additional meaningful integration tests, or another justified engineering improvement. These are **examples, not an exhaustive checklist**. Bonus work must be verifiable, may be examined during the demonstration and cannot compensate for missing core functionality.

A graphical interface is preferred but **does not itself earn bonus credit**. A well-implemented interactive TUI remains sufficient for the full core project.

7 Final Report and Submission

Submit a revised **PDF report** and an accessible **GitHub repository** containing the complete implementation. Your report must build on Practical 6, and should include:

- Team information, contribution statement and GitHub link.
- Workflow research with at least four credible references, assumptions and TIVIDY boundaries.
- Both industries: processes, participants, branching, failure paths and outcomes.
- Updated requirements, UML diagrams, pattern roles and justifications, where changes were needed after Practical 6.
- Implementation explanation, design decision history and links from important requirements to classes, patterns and tests.
- Build/run/test instructions and relevant Git and engineering evidence.

The repository must contain source and build files, the test suite, necessary data, `README.md`, `docs/` and UML exports, the final PDF report under `report/`, and any optional engineering configuration. Retain the research and design history started in Practical 6. A Google Docs link may be included if your team used one, but the PDF is the assessed report.

8 In-person Demo: 2 and 3 November 2026

Each group has **30 minutes** to demonstrate the completed system in person. **All team members must attend**, understand the whole design and be ready to answer questions. Demonstrate using the submitted code and bring a reliable local working setup.

Time	Suggested demonstration structure
0–4 min	Explain the reusable workflow engine and introduce both industries.
4–12 min	Live demonstration of industry 1, including decisions and an exception.
12–20 min	Live demonstration of industry 2 using the same engine.
20–25 min	Explain important GoF collaborations, diagrams and automated tests.
25–30 min	Questions, code inspection and individual understanding.

Be prepared to show your build, tests, workflow execution, UML models, repository history and any claimed CI/CD or Docker bonus work. Markers may request unscripted interactions or inspect code. Slides or recorded demonstrations cannot replace a live working system.

9 Project Progress and Academic Integrity

All design work, code and documentation must comply with University of Pretoria academic integrity rules. Cite sources and third-party dependencies. Every team member must be able to explain the group's design and implementation.

Final reminder: One generic TIVIDY engine. Two genuinely different industries. Ten meaningful GoF patterns. A working interactive C++11 application. Real framework-based tests.